Attitude Determination of a Spacecraft using Simple Kalman Filtering

Anan Faiyaz Radin[1], Sadman Ishraq[2]

[1]North South University, Bangladesh

[2]North South University, Bangladesh

[1]anan.radin@northsouth.edu; [2]sadman.ishraq@northsouth.edu

***Abstract*— Deployment of precise and efficient spacecrafts to manoeuvre space environment requires reliable sensor fusion of active and redundant passive heterogenous or homogenous sets of sensor data in near real time. For real time state estimation or attitude determination, a widely adapted filtering technique known as the Kalman filter has been adopted industry wide for multiple decades. Multiple variations of the filter already exist. This paper approaches a method to develop a modular MATLAB package for simple Kalman Filtering of a Gyroscope and Accelerometer. New sensor data can be assimilated in the package in order to create reliable software simulation for pre-prototype development of a spacecraft.**

Keywords*— *Space Situational Awareness, Attitude Determination, Sensor Fusion, MATLAB, Kalman Filter

I. INTRODUCTION

Space Situational Awareness is a growing field of study owing to the rapid population of space environment. For rapid deployment of new spacecrafts or satellites into orbit for the testing and implementation of novel technology missions, a reliable methodology for prototyping Attitude Determination & Control Systems (ADCS) are highly sought after and required.

Some research quickly reveals that while there are many existing methodologies and Off-the-Shelf components worldwide, there is a lack of a ground up prototyping methodology for developing such systems for a satellite or spacecraft. This study takes a ground-up approach into developing such a ground-up Attitude Determination system for the spacecraft to accurately define the attitude state vector of the spacecraft. It thus facilitates a quick and prompt sensor sizing method during the preliminary design phase of a spacecraft.

MATLAB is chosen as the simulation platform due to its widespread availability and flexibility. The MATLAB simulation package is straight forward in its integration of sensors, and utilizes a simple Kalman Filter, with the option for seamless upgrade to custom variations of the Kalman filter as necessary.

II. LITERATURE REVIEW

Spacecraft Attitude: The orientation of a spacecraft, particularly its body axis frame with reference to another reference frame, usually the earth centred inertial frame, at any point in orbit.

After deployment, a spacecraft is immediately subject to tumbling and the harsh space environment. Due to being continuously subject to various disturbances, internal and external, the spacecraft will deviate from intended attitude as well as provide noisy sensor readings. It is thus necessary to accurately define and determine the attitude of a spacecraft so that it can be corrected or controlled as necessary in orbit.

The determination is achieved by implementing filters on the sensor readings to acquire accurate readings after a number of iterations. Spacecraft Attitude Determination requires lucid insight into the following categories:

A. Coordinate Frames
B. Orbital Mechanics
C. Rigid Body Kinematics
D. Sensors
E. Sensor Fusion
F. Sensor Fusion

A. Coordinate Frames [2]

By Sidi and others, a few common reference frames are industry standardized:

- Orbit Reference Frame or, Local Vertical Local Horizontal (LVLH) Reference System
- Body Fixed Frame (BR) or, Satellite's Axis Frame
- Earth Centred Inertial Frame (ECI) or, Geocentric-Inertial System(J2000) or, International Celestial Reference Frame (ICRF)
- Perifocal Coordinate System

B. Orbital Mechanics [2] [8]

Sidi & Vallado gives us the following 6 classical orbit parameters for any standard rigid body space vehicle,

i. The semimajor axis,	$a = (r_a+r_p)/2$
ii. The Eccentricity,	$e = (r_a-r_p)/(r_a+r_p)$
iii. The Inclination,	i
iv. The Righ Ascension of the Ascending Node ,	Ω
v. The Argument of Perigee,	ω
vi. The Mean Anomaly,	$M = n\,(t-t_0)$

The 6 are combinedly defined by the *The State Vector,* $[\alpha] = [\ a\ \ e\ \ i\ \ \Omega\ \ \omega\ \ M\ ]^T$

Defining the parameters allows us to then define the LaGrange's Equations for Non-Keplerian Orbits:

$$\frac{\mathrm{da}}{\mathrm{dt}} = \frac{2}{\mathrm{na}}.\left(\frac{\partial U}{\partial M}\right)$$

$$\frac{\mathrm{de}}{\mathrm{dt}} = \left\{\frac{1-e^2}{\mathrm{na}^2 e}.\left(\frac{\partial U}{\partial M}\right)\right\} - \left\{\frac{\sqrt{1-e^2}}{\mathrm{na}^2 e}.\left(\frac{\partial U}{\partial \omega}\right)\right\}$$

$$\frac{\mathrm{di}}{\mathrm{dt}} = \frac{-1}{\mathrm{na}^2\sqrt{1-e^2}\sin(i)}.\left\{\left(\frac{\partial U}{\partial \Omega}\right) + \cos(i).\left(\frac{\partial U}{\partial \omega}\right)\right\}$$

$$\frac{\mathrm{d}\Omega}{\mathrm{dt}} = \frac{1}{\mathrm{na}^2\sqrt{1-e^2}\sin(i)}.\left\{\left(\frac{\partial U}{\partial i}\right)\right\}$$

$$\frac{\mathrm{d}\omega}{\mathrm{dt}} = \left\{\frac{\sqrt{1-e^2}}{\mathrm{na}^2 e}.\left(\frac{\partial U}{\partial e}\right)\right\} - \left\{\left(\frac{\cos(i)}{\mathrm{na}^2\sqrt{1-e^2}\sin(i)}\right).\left(\frac{\partial U}{\partial i}\right)\right\}$$

$$\frac{\mathrm{dM}}{\mathrm{dt}} = n - \{\frac{2}{\mathrm{na}}.(\frac{\partial U}{\partial a})\} - \{\frac{\sqrt{1-e^2}}{\mathrm{na}^2 e}.(\frac{\partial U}{\partial e})\}$$

C. Rigid Body Kinematics

The rotation matrices relating various reference frames in 3D is covered under Rigid Body Kinematics. There are 5 well recognized methods. All of them represent the rotation of the spacecraft body frame with respect to a defined reference frame. However, each has its perks and downfalls in terms of computation and complexity. They are:

i. Directional Cosine Matrix (DCM)
ii. Euler Axis & Principal Angle
iii. Euler Angles
iv. Quaternion
v. Gibbs or Rodrigues Parameters

As found in Sidi, Regan, Zipfel, Tewari & Markeley [2] [7]; there exists well defined transformations in between all the 5 axes, allowing quick conversions between them for smoother calculations. The most straightforward and intuitive axis method are the Euler angles, aka, The Yaw (Psi), The Pitch (Theta) and The Roll (Phi). However, in terms of complexity and computation considerations, the Quaternions, a 4x1 Column Vector, also known as the Euler Parameter Vector is the most efficient.

Euler Angles to Quaternions and vice versa can be achieved by the following MATLAB functions as found in [10].

```
function e = EP2Euler321(q)
function q = Euler3212EP(e)
```

D. Sensors

Determination demands perception of the environment we are operating in. Attitude determination in the space environment majorly requires exteroceptive passive sensors. 4 primary types of sensors are utilized in a space vehicle:

i. Gyroscopes:
It provides the angular rates about the body-fixed frame,

$$\omega = [\ \omega_x\ \ \omega_y\ \ \omega_z\ ]^T$$

ii. Accelerometer:
It provides the total inertial acceleration of the body subtracting gravity. It is usually placed at the centre of gravity of the body. Here, Output = Inertial Accel of Body- Gravity

$$\bar{f} = N.\frac{d\bar{v}}{\mathrm{dt}} - \bar{g}$$

iii. Sun Sensors:
It provides a unit vector to the sun, $\hat{S}$, in the body-fixed frame. A coarse sun sensor generates a current proportional to the incident sunlight.

$$I(\alpha) = I(0).\cos(\alpha)$$
$$\hat{S}.\hat{n} = \cos(\alpha)$$

iv. Magnetometer:
It is similar to a 3D compass, measures the magnetic field direction, $\hat{m}_B$.

The basic nominal accuracies for the sensors can be found in [4],

TABLE I
SENSOR ACCURACY & APPLICATIONS

Sensor	**Accuracy**	**Characteristics & Applicability**
Magnetometers	1.0 Deg (5000km Alt) 5.0 Deg (200 km Alt)	Attitude measured relative to Earth's local magnetic field. Magnetic field uncertainties and variability dominate accuracy.
Earth Sensors	0.05 Deg (GEO) 0.1 Deg (LEO)	Horizon uncertainties dominate earth accuracy.
Sun Sensors	0.01 Deg	Typical FOV +-30 deg
Star Sensors	2 Arc-Sec	Typical FOV +-6 deg
Gyroscopes	0.001 deg/hr	Normal use involves periodically resetting reference
Directional Antennas	0.01 to 0.5 Deg	Typically, 1% of the antenna bandwidth

E. Sensor Fusion

The use of multiple sensors to get multiple simultaneous or timed readings about the same state is recognized in the field as sensor fusion. Alternatively, multi-sensor systems can address these challenges through redundancy of individual sensors through sensor diversity (e.g. using sensors with different characteristics to offset limitations of others). Detailed breakdown is cover by Joseph Lorenzetti in [5].

F. Localization and Filtering

Compared to other forms of filters already in use, e.g. Moving Average Filter, Low Pass Filter, Bayes Filter etc. the Kalman Filter provides the maximum flexibility and versatility in terms of upgradation, robustness as well as the option for sensor fusion. To further compensate performance, variations of the KF has also been developed like the Extended Kalman Filter (EKF, Unscented Kalman Filter (UKF), Multiplicative Extended Kalman Filter (MEKF) and so on. Detailed study is found in [3]–[5].

III. **SIMPLE KALMAN FILTER**

The Kalman filter [3]–[5], [9] is an algorithm for state estimation that leverages the Gaussian distribution for efficiency and can be applied to continuous state models. The assumption is made that both the state transition and measurement models are linear with Gaussian noise.

A simple Kalman filter deals with a linear state model. The 4 steps for implementing a simple Kalman Filter can be best visualized with the diagram below:

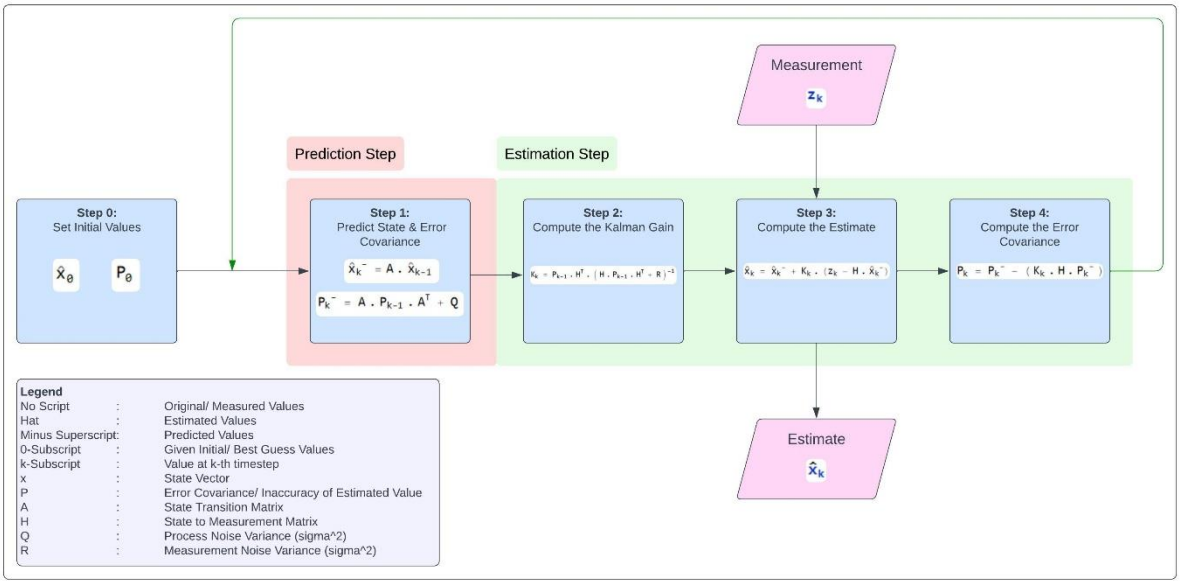

Fig. 1 Flowchart for Simple Kalman Filtering of Linear Models

The linear model for the setup is defined as,

System State Model : $x_{k+1} = A \cdot x_k + w_k$: Linear System Model + Process Noise

Measurement Model : $z_{k+1} = H \cdot z_k + v_k$: Linear Sensor Model + Measurement Noise

x_k	= State Vector at k-th timestep	(nx1 column vector)
z_k	= Measurement Vector	(mx1 column vector)
A	= State Transition Matrix	(nxn matrix)
H	= State to Measurement Matrix	(mxn matrix)
W_k	= State Transition Noise/ Environmental	(nx1 column vector)
V_k	= Measurement Noise/ Default Sensor Noise	(mx1column vector)
Covariance of the noise:		
Q	= Covariance matrix of w_k	(nxn diagonal matrix)
R	= Covariance matrix of v_k	(mxm diagonal matrix)

If n>1, W_k = {wk1, wk2, …. wkn} represents the noises for {xk1, xk2, ….., xkn}; thus gives the variances {sigma1squared, sigma2squared……, sigmaNsquared} respectively for each. For R, usually provided by the sensor manufacturer specifications. But it can often deviate.

So, both Q& R are mostly determined through trial-error and calibration. We get better performance from the Kalman filter if these 2 matrices are closer to the real-world noise/ covariance. Say, if for a sensor, given, +-2 m → then, Sigma^2 =4 (m^2).

The best approach is to overestimate the error and observe how the Filter performs and fine tune the values from there for the best performance.

IV. MATLAB IMPLEMENTATION

We firstly derive the mathematical foundation [3], [9] for our software implementation.

A. MATLAB System Model

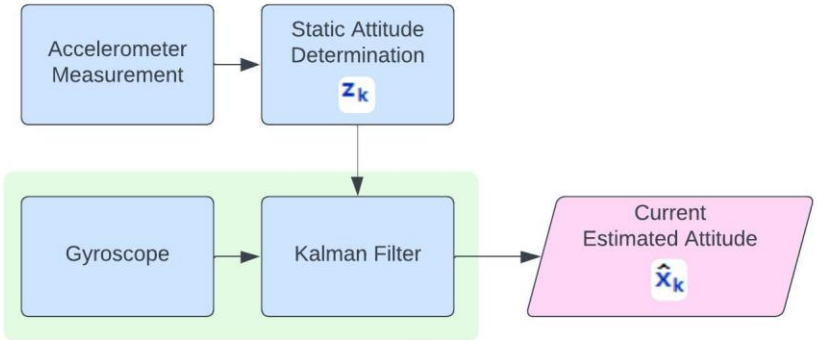

Fig. 2 Flowchart for MATLAB system model

B. MATLAB Setup

Our state vector is, $x = [\ \psi\ \ \theta\ \ \phi\ ]^T$ in Euler Angles, and in Quaternions, $x_k = [\ Q_0\ \ Q_1\ \ Q_2\ \ Q_4\ ]_k{}^T$

Expressing in terms of the linear Kalman model form, $\hat{x}_k{}^- = A.\hat{x}_{k-1}$, we get,

$$\begin{pmatrix} Q_0 \\ Q_1 \\ Q_2 \\ Q_3 \end{pmatrix}_{k+1} = (\mathbb{I}_{4\,x4} + \Delta t.\,B)\,.\begin{pmatrix} Q_0 \\ Q_1 \\ Q_2 \\ Q_3 \end{pmatrix}_k$$

,where,

$$B = \begin{pmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_2 & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_2 & \omega_x & 0 \end{pmatrix}$$

and, $\omega = [\ \omega_x\ \ \omega_y\ \ \omega_z\ ]^T$, from the gyroscope readings.

Again, for measurement, we get, $z_k = [\ \psi\ \ \theta\ \ \phi\ ]^T$, which is the measurement input.

This is derived from the accelerometer output, $\bar{f} = [f_1\ \ f_2\ \ f_3]^T$, as follows:

$$\theta = \sin^{-1}(\frac{f_1}{g}) \qquad \text{and, } \phi = \sin^{-1}(\frac{-f_2}{g\cos(\theta)})$$

That is, by simple sensor fusion of feeding the accelerometer readings as input measurements to the Kalman Filter already using the gyroscope readings, we get our desired output. Note- The f3 readings for Yaw is ignored for simplicity's sake.

The noise covariances are,

$Q = 0.0001.\mathbb{I}_{4x4}$ and, $R = 10.\mathbb{I}_{4x4}$

The initial states are,

$$(\psi, \theta, \phi) = (0,0,0) \quad => \quad x_0{}^- = [\ 1\ \ 0\ \ 0\ \ 0\ ]^T$$

$$P_0{}^- = \mathbb{I}_{4x4}$$

The sample data files are Accel.mat and Gyro.mat available at [11].

C. Summarized MATLAB Code

```
Nsamples = 41500; % Number of data points
EulerSaved = zeros(Nsamples, 3);
dt = 0.01; % seconds of time-step
psi = 0 ; % initial yaw guess
theta=0 ; %pitch
phi = 0 ; %roll
for k=1:Nsamples
  [w1 w2 w3] = GetGyro();  % load gyro data w1, w2, w3
  A = eye(4) + dt*1/2*[ 0   -w1  -w2  -w3 ;
                        w1   0    w3  -w2 ;
                        w2  -w3   0    w1 ;
                        w3   w2  -w1   0  ];
  [f1 f2] = GetAccel(); % load accelerometer data
  [phi theta] = EulerAccel(f1, f2); % radians
  z = Euler3212EP([ psi theta phi ]'); %z is the euler params
  [psi theta phi] = EulerKalman(A, z); % radians
  EulerSaved(k, :) = [ psi theta phi ];
end
PsiSaved   = EulerSaved(:, 1) * 180/pi;
ThetaSaved = EulerSaved(:, 2) * 180/pi;
PhiSaved   = EulerSaved(:, 3) * 180/pi;
t = 0:dt:Nsamples*dt-dt;


function [psi theta phi] = EulerKalman(A, z)
persistent H Q R
persistent x P
persistent firstRun
if isempty(firstRun)
  H = eye(4);
  Q = 0.0001*eye(4);
  R = 10*eye(4);
  x = [1 0 0 0]';
  P = 1*eye(4);
  firstRun = 1;
end
xp = A*x;
Pp = A*P*A' + Q;
K = Pp*H'*inv(H*Pp*H' + R);
x = xp + K*(z - H*xp);
P = Pp - K*H*Pp;
e=EP2Euler321(x); % Euler param. to yaw-pitch-roll
psi  =e(1); % yaw [rad]
theta=e(2); % pitch [rad]
phi  =e(3); % roll [rad]
end
function [phi theta] = EulerAccel(ax, ay)
g = 9.8;
theta = asin(ax / g );
phi   = asin(-ay / (g*cos(theta)));
end
function [ax ay az] = GetAccel()
persistent fx fy fz
persistent k firstRun
if isempty(firstRun)
  load Accel
  k = 1;
  firstRun = 1;
end
ax = fx(k);
ay = fy(k);
az = fz(k);
k = k + 1;
end
function [p q r] = GetGyro()
persistent wx wy wz
persistent k firstRun
if isempty(firstRun)
  load Gyro
  k = 1;
  firstRun = 1;
end
p = wx(k);
q = wy(k);
r = wz(k);
k = k + 1;
end
```

Full MATLAB package is available at [11].

V. RESULTS

We run the MATLAB package, "TestEuler" and "TestKalman" for our results,

A. Attitude Estimation using Euler Integration

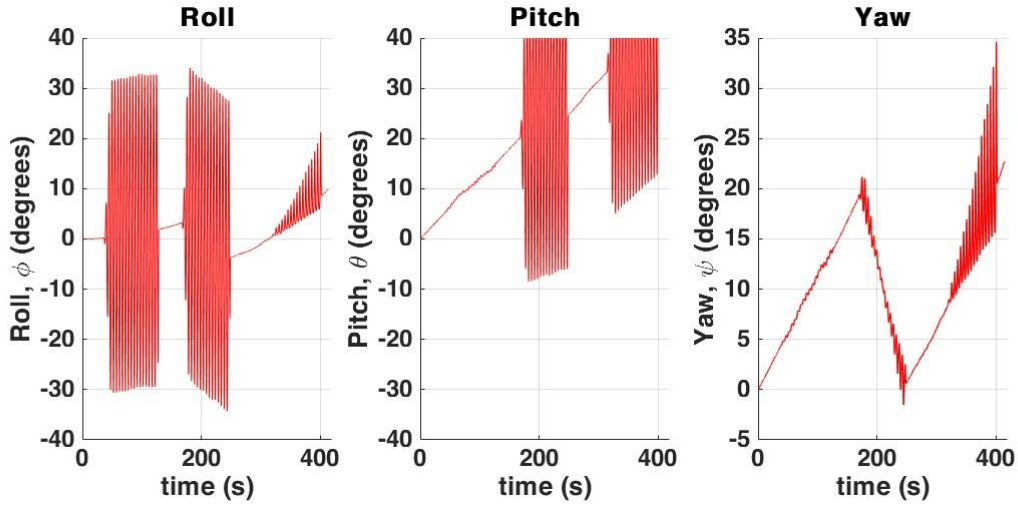

Fig. 3 Attitude Determination using Gyroscopes readings and Direct Euler Integration

B. Attitude Estimation using Simple Kalman Filter

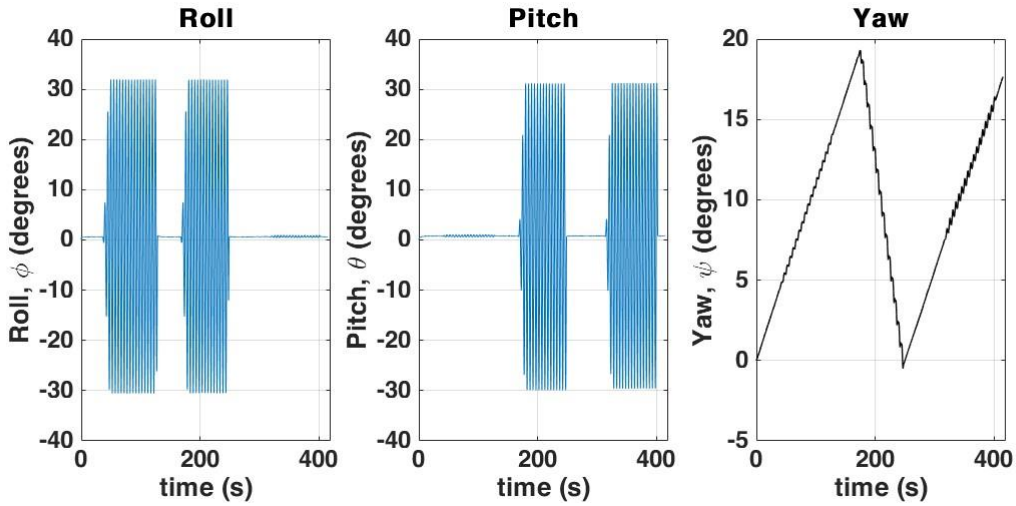

Fig. 4 Attitude Determination using Gyroscope & Accelerometer readings with sensor fusion in simple Kalman Filter

VI. CONCLUSIONS

By comparing our results, it is evident, that a Kalman Filter does a splendid job of outputting clean and noise free data. And by observation, the computation time for Kalman filter is far lower than for Euler Integration.

Going forward, the pointing modes for space vehicle may be introduced to create a MATLAB package for implementing any N number of Sun, Magnetometer, or IMU sensors for even better attitude determination.

ACKNOWLEDGEMENT

The authors would like to express their heartfelt gratitude towards their project and research supervisor, Dr. Shahnewaz Siddique, Assistant Professor, Department of Electrical and Computer Engineering, North South University, Bangladesh, for his invaluable support, precise guidance, and advice pertaining to the experiments, research and theoretical studies carried out during the course of the current project and also in the preparation of the current report.

REFERENCES

[1] J. R. Wertz, D. F. Everett, and J. J. Puschell, *Space Mission Engineering: The New SMAD*. 2011.

[2] M. J. Sidi, *Spacecraft Dynamics and Control*. 1997. doi: 10.1017/cbo9780511815652.

[3] P.-Y. Kim and P. Kim, *Kalman filter for beginners: With MATLAB Examples*. Createspace Independent Publishing Platform, 2011.

[4] H. Schaub and J. L. Junkins, *Analytical Mechanics of Space Systems, Fourth Edition*. 2018. doi: 10.2514/4.105210.

[5] "Principles of Robot Autonomy." Accessed: Jun. 20, 2024. [Online]. Available: https://stanfordasl.github.io/PoRA-I/aa174a_aut2324/pdfs/AA_PoRA_lecture_notes.pdf

[6] B. Wie, *Space Vehicle Dynamics and Control*. AIAA (American Institute of Aeronautics & Astronautics), 2008.

[7] F. L. Markley and J. L. Crassidis, *Fundamentals of spacecraft attitude determination and control*. Springer, 2014.

[8] D. A. Vallado, *Fundamentals of Astrodynamics and Applications*. Springer Science & Business Media, 2001.

[9] "Space Vehicle Dynamics." Accessed: Jun. 20, 2024. [Online]. Available: https://www.youtube.com/playlist?list=PLUeHTafWecAXDFDYEwunLL2V2kwqKzkvJ

[10] Dangpzanco, "GitHub - dangpzanco/attitude: Attitude dynamics functions for MATLAB," *GitHub*. https://github.com/dangpzanco/attitude/tree/master

[11] Xylomylo, "GitHub - xylomylo/kalman-filter-for-attitude-determinationEEE494," *GitHub*. https://github.com/xylomylo/kalman-filter-for-attitude-determinationEEE494

[12] Xylomylo, "Supplementary Material: Wolfram Mathematica notebook file for equations used in the report," *GitHub*. https://github.com/xylomylo/kalman-filter-for-attitude-determinationEEE494/blob/main/KF.nb (accessed Jun. 20, 2024).